SIMATS ENGINEERING

ASSESSMENT TOOL 1: Knowledge Representation Design Assessment

COURSE NAME: ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS **COURSE CODE: MLA01**

COURSE OUTCOME COVERED

CO3: *Implement propositional and first-order logic using resolution, unification, and inference techniques for knowledge representation and reasoning.*(BTL 3)

Assessment Tool Weightage: 40 %

Total Marks: 100 **Time: 60 Mins**

No. of Questions: 5

Annexure A

Knowledge Representation Design Assessment

Code of Conduct:

*I (*V Sunil Kumar / 192425292 *) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:
V Sunil Kumar

Criterion	Weightage	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Knowledge Engineering and Knowledge Base Design	15%	
3. Propositional Logic Representation	10%	
4. First-Order Logic Representation	15%	
5. Unification and Resolution	15%	
6. Forward and Backward Chaining	15%	
7. Inference and Reasoning Accuracy	10%	
8. System Architecture / Representation Diagram	5%	
9. Prototype Implementation and Results	5%	
Total	100%	

Signature of the Faculty Total Marks Awarded

ANSWER ANY ONE QUESTION

1. Smart Healthcare Diagnosis System

1. Problem Analysis and Domain Understanding

Problem Statement

A hospital wants to develop an AI-based Smart Healthcare Diagnosis System that can identify possible illnesses from patient symptoms. The system uses a knowledge base containing patient facts and medical rules. Logical inference techniques are used to derive the patient's possible condition and recommend an appropriate action.

Objective

The main objectives of the system are:

1. To represent patient symptoms using logical statements.
2. To construct a structured knowledge base containing facts and rules.
3. To represent the knowledge using Propositional Logic and First-Order Logic.
4. To apply unification and substitution.
5. To use forward chaining to derive Ravi's condition.
6. To use backward chaining to determine whether Ravi needs rest.
7. To use resolution to prove a selected query.
8. To produce a logically valid final inference.

Entities

Entity	Description
Ravi	Patient
Fever	Patient symptom
Cough	Patient symptom
Body Pain	Patient symptom
Flu	Possible illness
Viral Infection	Possible illness
Rest	Recommended action

Given Facts

- Ravi has fever.
- Ravi has cough.
- Ravi has body pain.

Given Rules

- Patients with fever and cough may have flu.
- Patients with flu and body pain may have viral infection.

- Patients with viral infection should be advised to rest.

The expected inference is:

Symptoms → Flu → Viral Infection → Rest

---

2. Knowledge Engineering and Knowledge Base Design

The knowledge base consists of two major components:

A. Facts

Facts represent information that is already known.

Fever(Ravi)

Cough(Ravi)

BodyPain(Ravi)

B. Rules

Rules represent relationships between facts.

Fever(x) AND Cough(x) → Flu(x)

Flu(x) AND BodyPain(x) → ViralInfection(x)

ViralInfection(x) → Rest(x)

Complete Knowledge Base

FACTS:

Fever(Ravi)

Cough(Ravi)

BodyPain(Ravi)

RULE 1:

Fever(x) ∧ Cough(x) → Flu(x)

RULE 2:

Flu(x) ∧ BodyPain(x) → ViralInfection(x)

RULE 3:

ViralInfection(x) → Rest(x)

The knowledge base is consistent because each derived conclusion is supported by the given facts and rules.

---

3. Propositional Logic Representation

In propositional logic, each statement is represented by a proposition.

Let:

F = Ravi has fever

C = Ravi has cough

B = Ravi has body pain

FL = Ravi has flu

V = Ravi has viral infection

R = Ravi should be advised to rest

Facts

[

F

]

[

C

]

[

B

]

Rules

Rule 1: Fever and cough imply flu

[

(F \land C) \rightarrow FL

]

Rule 2: Flu and body pain imply viral infection

[

(FL \land B) \rightarrow V

]

Rule 3: Viral infection implies rest

[

V \rightarrow R

]

Propositional Inference

[

F,C,B

]

[

(F\land C)\rightarrow FL

]
Therefore:
[
FL
]
Then:
[
(FL\land B)\rightarrow V
]
Therefore:
[
V
]
Finally:
[
V\rightarrow R
]
Therefore:
[
\boxed{R}
]
Hence, Ravi should be advised to rest.

---

4. First-Order Logic Representation

First-Order Logic provides a more general representation by using predicates, variables, constants and quantifiers.

Predicates

Fever(x)

Cough(x)

BodyPain(x)

Flu(x)

ViralInfection(x)

Rest(x)

Where x represents a patient.

Constant

Ravi

represents the particular patient Ravi.

Facts

[
Fever(Ravi)
]

[
Cough(Ravi)
]

[
BodyPain(Ravi)
]

Rules

Rule 1

For every patient, if the patient has fever and cough, then the patient may have flu.

[
\forall x[(Fever(x)\land Cough(x))\rightarrow Flu(x)]
]

Rule 2

For every patient, if the patient has flu and body pain, then the patient may have viral infection.

[
\forall x[(Flu(x)\land BodyPain(x))\rightarrow ViralInfection(x)]
]

Rule 3

For every patient, if the patient has viral infection, then the patient should rest.

[
\forall x[ViralInfection(x)\rightarrow Rest(x)]
]

FOL Inference

[
Fever(Ravi)\land Cough(Ravi)
]

gives:

[
Flu(Ravi)
]

Then:

[
Flu(Ravi)\land BodyPain(Ravi)
]
gives:
[
ViralInfection(Ravi)
]
Then:
[
ViralInfection(Ravi)
]
gives:
[
Rest(Ravi)
]

---

5. Unification and Substitution

Unification is the process of finding substitutions that make two logical expressions identical.

Unification Example 1

Consider:
[
Fever(x)
]
and
[
Fever(Ravi)
]
Substitution:
[
{x/Ravi}
]
After substitution:
[
Fever(Ravi)=Fever(Ravi)
]
Therefore, they successfully unify.

The **Most General Unifier (MGU)** is:

[
\boxed{{x/Ravi}}
]

Unification Example 2

Consider:

[
Flu(x)
]

and

[
Flu(Ravi)
]

Substitution:

[
{x/Ravi}
]

After substitution:

[
Flu(Ravi)=Flu(Ravi)
]

Therefore, they successfully unify.

MGU:

[
\boxed{{x/Ravi}}
]

---

6. Forward Chaining

Forward chaining is a **data-driven inference technique**. It starts with known facts and applies rules to derive new facts until the required conclusion is obtained.

Initial Facts

Fever(Ravi)

Cough(Ravi)

BodyPain(Ravi)

Step 1

Apply Rule 1:

[
Fever(x)\land Cough(x)\rightarrow Flu(x)
]
Substitute:
[
x=Ravi
]
Therefore:
[
Fever(Ravi)\land Cough(Ravi)
\rightarrow Flu(Ravi)
]
New fact:
Flu(Ravi)

Step 2

Apply Rule 2:
[
Flu(x)\land BodyPain(x)\rightarrow ViralInfection(x)
]
Substitute:
[
x=Ravi
]
Therefore:
[
Flu(Ravi)\land BodyPain(Ravi)
\rightarrow ViralInfection(Ravi)
]
New fact:
ViralInfection(Ravi)

Step 3

Apply Rule 3:
[
ViralInfection(x)\rightarrow Rest(x)
]
Substitute:
[

x=Ravi

]

Therefore:

Rest(Ravi)

Forward-Chaining Sequence

Fever(Ravi)

+

Cough(Ravi)

↓

Flu(Ravi)

+

BodyPain(Ravi)

↓

ViralInfection(Ravi)

↓

Rest(Ravi)

Forward Chaining Result

[

\boxed{ViralInfection(Ravi)}

]

and

[

\boxed{Rest(Ravi)}

]

---

7. Backward Chaining

Backward chaining is a **goal-driven inference technique**. It starts with the required goal and works backward to determine whether the goal can be supported by known facts.

Query

[

Rest(Ravi)

]

Step 1

Find a rule that concludes Rest(x):

[

ViralInfection(x)\rightarrow Rest(x)

]

Therefore, we need to prove:

[

ViralInfection(Ravi)

]

Step 2

Use Rule 2:

[

Flu(x)\land BodyPain(x)\rightarrow ViralInfection(x)

]

Therefore, we need:

[

Flu(Ravi)

]

and

[

BodyPain(Ravi)

]

BodyPain(Ravi) is already a fact.

Step 3

To prove Flu(Ravi), use Rule 1:

[

Fever(x)\land Cough(x)\rightarrow Flu(x)

]

We already know:

[

Fever(Ravi)

]

and

[

Cough(Ravi)

]

Therefore:

[

Flu(Ravi)

]

Step 4

Thus:

[
Flu(Ravi)+BodyPain(Ravi)
\rightarrow ViralInfection(Ravi)
]

and:

[
ViralInfection(Ravi)
\rightarrow Rest(Ravi)
]

Backward-Chaining Proof

Goal: Rest(Ravi)

↑

ViralInfection(Ravi)

↑

Flu(Ravi) + BodyPain(Ravi)

↑

Fever(Ravi) + Cough(Ravi)

↑

Known Facts

Therefore:

[
\boxed{Rest(Ravi)}
]

is successfully proved.

---

8. Resolution

We select the query:

[
Rest(Ravi)
]

For resolution, the query is negated:

[
\neg Rest(Ravi)
]

We convert the relevant rules into clause form.

Rule 1

$$(Fever(x)\land Cough(x))\rightarrow Flu(x)$$

becomes:

$$\neg Fever(x)\lor\neg Cough(x)\lor Flu(x)$$

For Ravi:

$$\neg Fever(Ravi)\lor\neg Cough(Ravi)\lor Flu(Ravi)$$

Rule 2

$$(Flu(x)\land BodyPain(x))\rightarrow ViralInfection(x)$$

becomes:

$$\neg Flu(x)\lor\neg BodyPain(x)\lor ViralInfection(x)$$

For Ravi:

$$\neg Flu(Ravi)\lor\neg BodyPain(Ravi)\lor ViralInfection(Ravi)$$

Rule 3

$$ViralInfection(x)\rightarrow Rest(x)$$

becomes:

$$\neg ViralInfection(x)\lor Rest(x)$$

For Ravi:

$$\neg ViralInfection(Ravi)\lor Rest(Ravi)$$

Resolution Step 1

Known fact:

[
Fever(Ravi)
]

Resolve with:

[
\neg Fever(Ravi)\lor\neg Cough(Ravi)\lor Flu(Ravi)
]

Result:

[
\neg Cough(Ravi)\lor Flu(Ravi)
]

Using the fact:

[
Cough(Ravi)
]

we obtain:

[
Flu(Ravi)
]

Resolution Step 2

Use:

[
Flu(Ravi)
]

and:

[
\neg Flu(Ravi)\lor\neg BodyPain(Ravi)\lor ViralInfection(Ravi)
]

Together with:

[
BodyPain(Ravi)
]

we derive:

[
ViralInfection(Ravi)
]

Resolution Step 3

Use:

$$
\text{ViralInfection(Ravi)}
$$

and:

$$
\neg \text{ViralInfection(Ravi)} \lor \text{Rest(Ravi)}
$$

Therefore:

$$
\text{Rest(Ravi)}
$$

Resolution with Negated Query

We initially assumed:

$$
\neg \text{Rest(Ravi)}
$$

But the knowledge base derives:

$$
\text{Rest(Ravi)}
$$

Hence:

$$
\text{Rest(Ravi)} \land \neg \text{Rest(Ravi)}
$$

produces the empty clause:

$$
\boxed{\Box}
$$

The empty clause proves that the negated query is contradictory.

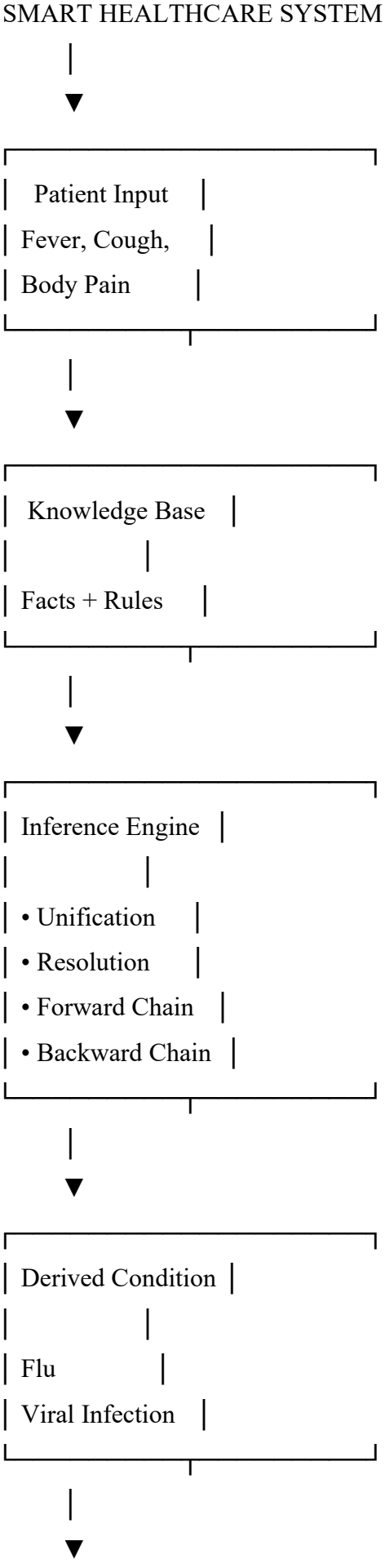
Therefore:

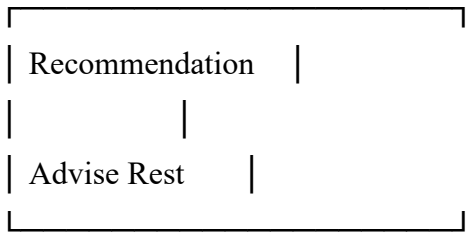
$$
\boxed{\text{Rest(Ravi)}}
$$

is logically entailed by the knowledge base.

---

9. System Architecture / Knowledge Representation Diagram





The architecture contains the **knowledge base, inference engine, reasoning techniques, and final output**, matching the assessment requirement for a complete and well-labelled representation diagram.

---

10. Final Inference and Reasoning Accuracy

The complete reasoning process is:

[
Fever(Ravi)+Cough(Ravi)
]
[
\Downarrow
]
[
Flu(Ravi)
]

Then:

[
Flu(Ravi)+BodyPain(Ravi)
]
[
\Downarrow
]
[
ViralInfection(Ravi)
]

Finally:

[
ViralInfection(Ravi)
]
[
\Downarrow
]

[
Rest(Ravi)
]

Final Inference

[
\boxed{\text{Ravi may have a viral infection and should be advised to rest.}}
]

The conclusion is logically supported by the facts and rules in the knowledge base. The inference engine does not independently diagnose a real medical condition; it derives the stated conclusion according to the rules supplied to the system.

---

11. Comparison of Inference Techniques

Technique	Starting Point	Application in System	Result
Propositional Logic	Statements	Represents Ravi's symptoms and conclusions	Represents knowledge
FOL	Predicates and variables	Represents general patient rules	Generalized reasoning
Unification	Predicates	Matches x with Ravi	Enables rule application
Forward Chaining	Known facts	Fever + Cough → Flu → Viral Infection	Derives conclusion
Backward Chaining	Goal	Rest → Viral Infection → Flu → Symptoms	Proves goal
Resolution	Clauses	Proves Rest(Ravi) using contradiction	Query established

---

12. Conclusion

The Smart Healthcare Diagnosis System successfully demonstrates knowledge representation and logical reasoning.

The patient facts are represented using Propositional Logic and First-Order Logic. A structured knowledge base is created using facts, predicates, variables, constants, and rules. Unification is used to substitute Ravi for the variable x. Forward chaining derives the patient's possible condition from the available symptoms, while backward chaining starts with the goal of proving whether Ravi needs rest. Resolution further validates the query by deriving a contradiction from its negation.

The final inference is:

$$
\boxed{Fever(Ravi)+Cough(Ravi)\rightarrow Flu(Ravi)}
$$

$$
\boxed{Flu(Ravi)+BodyPain(Ravi)\rightarrow ViralInfection(Ravi)}
$$

$$
\boxed{ViralInfection(Ravi)\rightarrow Rest(Ravi)}
$$

Therefore, **the system concludes that Ravi may have a viral infection and should be advised to rest**.

This answer covers the major rubric areas: problem understanding, knowledge-base design, logic representation, unification and resolution, forward/backward chaining, inference accuracy, system architecture, and clear presentation. The rubric gives the highest performance level to answers that provide complete and logically accurate representations and step-by-step reasoning.

Evaluation Rubrics:

Criterion	Weightage (%)	Excellent	Good	Average	Poor
Problem Analysis and Understanding	15%	13–15% – Clearly understands and analyzes the real-world problem, objectives, entities, facts, constraints, and expected inference.	10–12% – Understands the problem with minor omissions.	7–9% – Shows partial understanding with some important elements missing.	0–6% – Poor or incorrect understanding of the problem.
Knowledge Engineering & Knowledge Base Design	15%	13–15% – Develops a complete, consistent, and well-structured knowledge base with appropriate facts, predicates, entities, and rules.	10–12% – Knowledge base is well designed with minor inconsistencies or omissions.	7–9% – Basic knowledge base is developed but several facts/rules are missing or unclear.	0–6% – Knowledge base is incomplete, inconsistent, or inappropriate.
Propositional & First-Order Logic Representation	20%	17–20% – Correctly represents the domain using propositions, predicates, variables, constants, quantifiers, and logical connectives with excellent accuracy.	13–16% – Logic representation is mostly correct with minor errors.	9–12% – Provides basic representations but has several logical or syntactic errors.	0–8% – Logic representation is incorrect, incomplete, or missing.
Unification & Resolution	15%	13–15% – Correctly performs substitutions, identifies the MGU, and applies resolution systematically to derive valid conclusions.	10–12% – Applies unification and resolution correctly with minor errors.	7–9% – Demonstrates partial understanding but misses important steps or contains errors.	0–6% – Unable to correctly apply unification or resolution.

Forward Chaining & Backward Chaining	15%	13–15% – Correctly applies both forward and backward chaining with clear step-by-step reasoning and appropriate rule selection.	10–12% – Correctly applies both methods with minor errors or omissions.	7–9% – Demonstrates partial understanding of one or both chaining methods.	0–6% – Incorrectly applies or fails to demonstrate chaining techniques.
Inference & Reasoning Accuracy	10%	9–10% – Produces logically valid conclusions and clearly explains how facts and rules lead to the final inference.	7–8% – Conclusions are mostly correct with minor reasoning gaps.	5–6% – Some correct inferences but reasoning is incomplete or unclear.	0–4% – Conclusions are incorrect, unsupported, or missing.
Knowledge Representation Diagram / System Architecture	5%	5% – Provides a complete, accurate, well-labelled diagram showing knowledge base, inference engine, reasoning process, and output.	4% – Diagram is mostly correct with minor omissions.	3% – Diagram is partially complete or lacks clarity.	0–2% – Diagram is missing or incorrect.
Originality, Critical Thinking & Presentation	5%	5% – Demonstrates excellent independent thinking, appropriate real-world application, comparison of reasoning approaches, and clear presentation.	4% – Demonstrates good reasoning and clear presentation.	3% – Shows limited critical thinking and basic presentation.	0–2% – Shows little originality, weak reasoning, or poor presentation.